

MicroHosted

Motor self-hosted de microVMs, y su aplicación como micro-gateway de aislamiento IoT

Qué es

MicroHosted es un motor de máquinas virtuales ligeras (microVMs) construido sobre Firecracker y Jailer, la misma tecnología con la que AWS ejecuta Lambda. Cada microVM es una máquina completa, con su propio kernel Linux, que arranca en menos de dos segundos desde cero (y en torno a 100 ms desde snapshot) y consume unas decenas de MB. Todo se controla desde una API REST: crear, ejecutar, clonar y destruir máquinas de forma programática.

Existen plataformas comerciales que ofrecen algo parecido como servicio en la nube. La diferencia de enfoque aquí es que todo es self-hosted: corre en hardware propio y los datos nunca salen de él. El uso más natural de un motor así es el sandboxing, ejecutar código o datos en los que no confías (análisis, agentes de IA, entornos de prueba) sin que puedan tocar el host, pero el motor es generalista.

Qué puede hacer hoy

- **Ciclo de vida** completo de microVMs por API: crear, listar, ejecutar comandos, destruir.
- **Snapshots y fork**: clonar una máquina viva (memoria + disco) en caliente, o levantar flotas enteras desde una misma imagen base.
- **Redes segmentadas**: firewall restrictivo por defecto, una VM solo puede hablar con lo que se le permite explícitamente, hasta el nivel de IP y puerto concretos.
- **Comunicación** host↔VM por vsock: inyectar y extraer archivos o lanzar comandos sin usar la red. El host nunca monta ni parsea los discos del guest.
- **Imágenes** copy-on-write: cientos de VMs comparten la misma base y cada una solo paga sus diferencias.
- **Límites** de CPU, memoria y procesos por VM, más observabilidad (consumo real, salud del host).
- **Portabilidad**: funciona en x86 y en ARM64 (validado en Raspberry Pi).

Densidad: más de 150 máquinas simultáneas en un equipo modesto

Con imágenes optimizadas, el coste por máquina es tan bajo que la densidad deja de ser el problema. En esta prueba de flota se desplegaron 174 microVMs completas y simultáneas en un único equipo, hasta agotar la RAM disponible: unos 34 MB de coste medio por unidad, con un tiempo de despliegue estable (la máquina 174 tardó prácticamente lo mismo que la primera) y todas vivas y respondiendo al final de la prueba.

```
===== RESULTADO (fleet) =====
Desplegados:                174
Motivo de parada:           suelo de RAM (480 MB < 500 MB)
Vivos (muestra):            sí
Despliegue 1º vs último:    1783 ms → 1856 ms
PSS última VM:              37032 kB
RAM: 6508 MB → 480 MB
Coste medio por unidad:     34 MB
Health: {"status":"ok","checks":[{"name":"kvm","ok":true},
{"name":"database","ok":true}, {"name":"store_writable",
"ok":true}, {"name":"disk_space","ok":true,"detail":"190
99 MiB free of 20480 MiB"}, {"name":"store_cow","ok":true
}, {"name":"firecracker","ok":true}]}
```

Prueba de flota: 174 microVMs simultáneas, ~34 MB por unidad, despliegue estable de principio a fin.

Hacia dónde apunta: un micro-gateway de aislamiento IoT

Esa densidad abre un uso concreto: el gateway IoT. Los gateways edge habituales procesan los datos de los sensores en contenedores, que comparten el kernel del host: si el parser de un sensor se compromete, el atacante está a un paso de la planta entera. La idea que estoy explorando es darle a cada sensor su propia microVM (quizás por redes) desechable: el host no expone puertos hacia los sensores ni interpreta ningún protocolo, y si un parser cae, lo que cae es una máquina de 34 MB sin camino hacia el host, que se regenera limpia desde snapshot en unos 100 ms. Merece estudiarse si este enfoque, sumado a una capa de trazabilidad sobre la API, encaja como pieza para superar controles NIS2 y dar uso industrial al motor como controlador de sensores: la normativa no pide solo aislamiento, sino *control demostrable*, contener el impacto y poder probarlo, y el aislamiento por zonas que plantea encaja con cómo los propios estándares industriales (IEC 62443) estructuran una planta.

Flujo end-to-end: lectura de un sensor Modbus (pull)

Para probarlo de punta a punta montamos el caso más común en OT: interrogar por polling a un sensor Modbus. El parser es un pequeño programa en C que corre dentro de la microVM.

1. La VM vive en una red dedicada cuyo único destino permitido es su sensor (una IP y un puerto). No puede hablar con el host, con otras VMs ni con internet.

```
curl -s localhost:8080/v1/networks | jq '[] | select(.name = "iot-modbus")'
{
  "name": "iot-modbus",
  "bridge": "mhbr7bf92ce",
  "subnet": "172.16.8.0/24",
  "gateway": "172.16.8.1",
  "egress": false,
  "allowed_egress": [
    {
      "ip": "192.168.0.15",
      "protocol": "tcp",
      "port": 5020
    }
  ],
  "intra": false,
  "created_at": "2026-07-07T13:48:36+02:00"
}
```

Red "iot-modbus": salida denegada por defecto; único egress permitido, el sensor 192.168.0.15:5020.

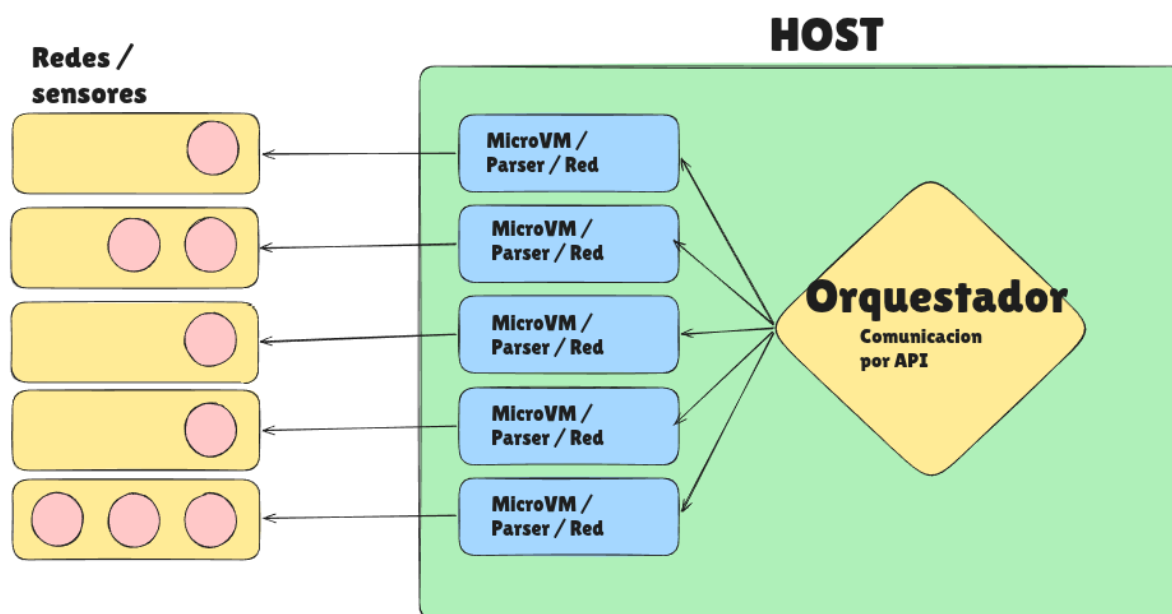
2. El parser se sube a la VM por vsock (también puede ir ya dentro de la imagen, de forma que la máquina nazca configurada) y se ejecuta: interroga al sensor, valida la trama y devuelve un JSON limpio por stdout, que llega al host por vsock, sin red de por medio.

```
curl -s -X PUT localhost:8080/v1/vms/b905dba8/files?path=parser --data-binary @/home/manu/MicroHosted/examples/modbus-pull/parser
```

Subida del parser a la microVM a través de la API de archivos (vsock).

Una línea que merece estudiarse: con NIS2 y la presión regulatoria sobre sectores críticos que dependen de IoT, el aislamiento fuerte y auto-alojado en el edge es un hueco que las plataformas basadas en contenedores no cubren por diseño. El propio sistema podría usarse para analizar con modelos de IA en ONNX dentro de MicroVms para analizar tráfico de red, detonar malware etc...

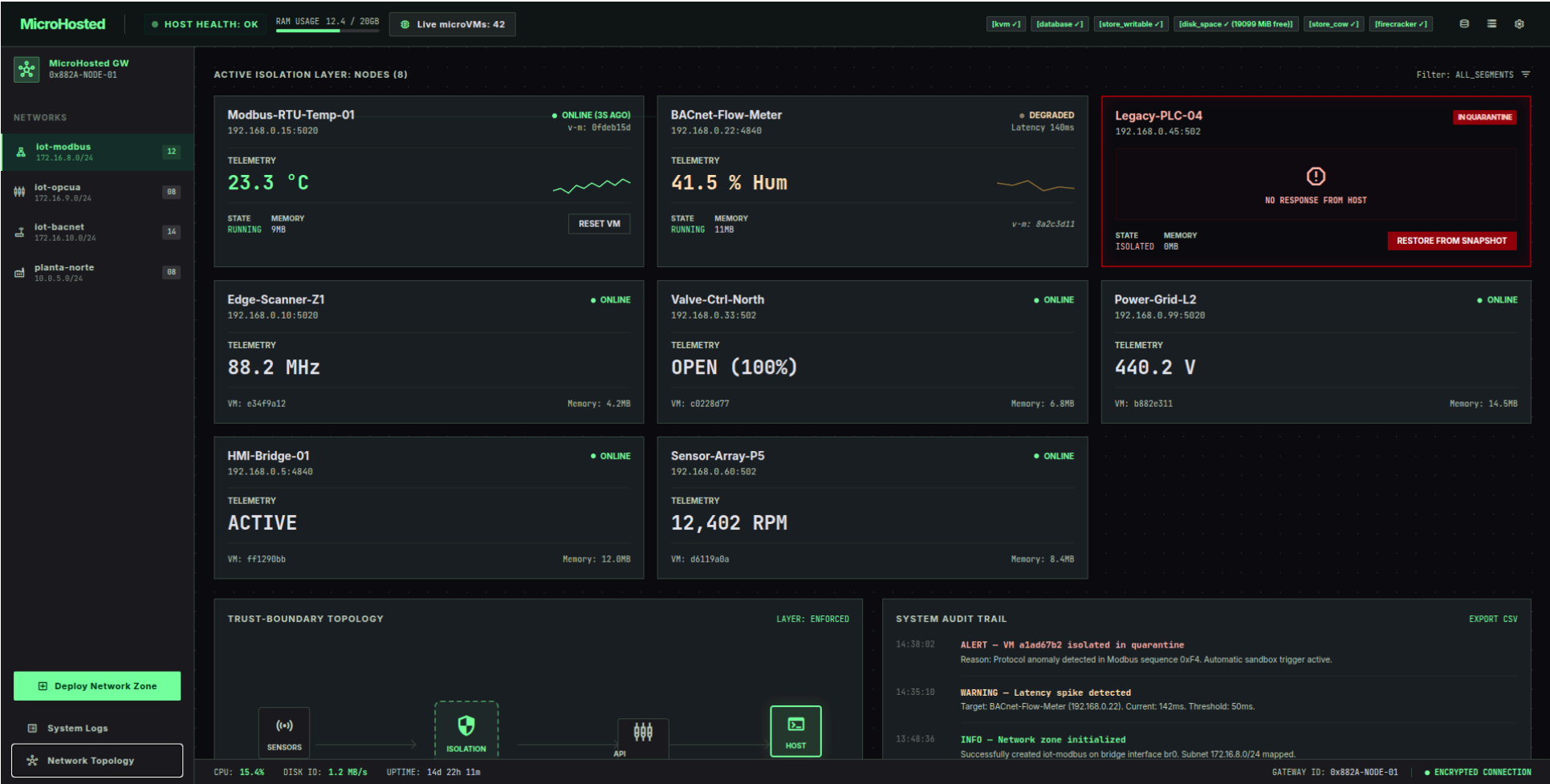
Futuro del proyecto



Ejemplo a Alto Nivel de cómo podría funcionar el sistema

Como el motor está prácticamente terminado y todo se controla desde una API REST, la separación entre núcleo e interfaz ya viene dada por diseño: cualquier capa gráfica se construye por encima sin tocar el motor. El trabajo pesado (aislamiento, redes segmentadas, snapshots, ciclo de vida) vive en el núcleo, y la interfaz se limita a orquestar llamadas a la API.

Sobre esa base, un panel de control aportaría dos cosas que hoy quedan en manos del operador por API. Primero, visibilidad: el estado de la última conexión de cada sensor, la salud del host, el consumo real de cada microVM y un mapa de qué máquina vive en qué red o segmento. Segundo, y decisivo para entornos regulados, trazabilidad: un registro auditable de eventos, cuándo se levantó una máquina, cuándo se aisló en cuarentena, qué anomalía la disparó, que convierte el aislamiento en *control demostrable*. Y la orquestación (levantar máquinas por sensor o por red, aislar, regenerar desde snapshot) pasa de ser una secuencia de llamadas a la API a una acción de un clic.



Ejemplo de Interfaz Web para la parte IOT